

课堂复盘与教学分析系统

M1 完成情况、后续开发计划与个人总结报告

Classroom Reflection and Teaching Analysis System

M1 Completion Status, Future Development Plan, and Individual Project Summary

第一部分：中文版

一、问题与用户

1.1 目标用户

本系统主要面向需要进行课后复盘的一线教师，当前支持以下课堂素材：

- 线下课堂录像；
- 直播网课录像；
- 录播课程视频。

M1 阶段主要服务单个教师对一节课进行完整复盘，暂不面向学校管理者、区域教研部门或大规模组织管理场景。

1.2 真实使用场景

教师完成一节课后，通常需要重新观看课堂视频，寻找课堂中的提问、等待、学生回答、教师反馈、连续讲授、表达密度和课堂结构等片段。

传统复盘方式主要存在以下困难：

- 完整课堂视频时间较长，人工反复拖动视频定位片段效率较低；
- 普通语音识别结果通常较碎，难以直接用于阅读和教学分析；
- 逐字稿、视频片段和复盘结论之间缺少稳定关联，教师难以快速核对分析结果；
- 通用语言模型容易根据局部内容作出过度推断；
- 线下课堂、直播网课和录播课程具有不同的可观察信息，不能使用完全相同的分析方式；

6. 教师需要具体、可执行的改进建议，而不是缺少依据的笼统评价。

1.3 最终结果与成功标准

M1 的目标是完成一条能够真实运行的单节课堂复盘流程：

上传课堂视频

- 生成带时间点逐字稿
- 教师修改逐字稿
- 计算课堂指标
- 生成候选教学证据
- 教师接受、修改或驳回
- 根据确认内容生成报告

M1 的成功标准包括：

- 视频能够真实上传并保存；
- 逐字稿由真实 ASR 服务生成；
- 逐字稿能够定位到视频时间点；
- 教师可以修改识别错误；
- 修改后的文本能够进入后续分析；
- 证据能够关联原文和视频位置；
- 教师可以接受、修改后接受或驳回证据；
- 最终报告只使用教师确认后的内容。

1.4 为什么只聚焦这一范围

本项目由我个人独立完成，开发时间、云服务预算、测试素材和工程资源均有限。

如果第一版同时建设多用户系统、学校管理后台、课件分析、视频画面识别、跨课堂比较、知识图谱和长期教师画像，会显著增加开发复杂度，使最核心的课堂复盘链路难以真正完成。

因此，M1 只聚焦一段真实课堂视频的完整处理过程，优先验证：

- 视频能否稳定进入系统；
- 逐字稿能否被教师阅读、定位和修改；
- 课堂指标和分析结果能否对应真实片段；
- 教师确认后能否形成可使用的报告。

这一范围能够直接解决教师复盘耗时、片段定位困难和分析结果不易核对的问题，也为后续两轮课堂比较和多课程管理建立基础。

二、方案与交互

2.1 核心任务流程

当前已经实现的主要任务流程为：

教师创建课堂

→ 选择课堂类型

→ 上传课堂视频

→ 视频和音频对象写入数据库

→ 后台任务开始处理

→ 调用真实 ASR

→ 保存带时间点逐字稿

→ Transcript Agent 整理逐字稿

→ 生成大段课堂记录

→ 教师修改并保存逐字稿

→ 教师确认校订完成

→ 计算基础课堂指标

→ 分析问题、等待、回答和反馈链

→ Teaching Evidence Agent 生成候选证据

→ 教师接受、修改后接受或驳回证据

→ 根据确认内容生成报告

当前主要通过结构化工作区完成课堂复盘。

AI 对话区尚未实现，因此当前版本暂不支持教师通过持续自然语言对话指挥分析流程、追问证据原因或要求系统重新处理指定片段。

2.2 当前工作区

当前工作区主要承担以下任务：

- 创建课堂；
- 选择课堂类型；
- 上传视频并显示真实上传进度；
- 播放课堂视频；
- 查看带时间点的逐字稿；

- 编辑和保存课堂大段文本；
- 点击翻译英文片段；
- 查看处理状态；
- 查看课堂指标；
- 查看候选教学证据；
- 跳转到证据对应的视频位置；
- 接受、修改或驳回证据；
- 查看最终报告。

播放器、逐字稿、指标卡和证据卡共同构成主要操作空间，避免只通过聊天文本展示复杂的课堂分析结果。

2.3 计划中的对话区与工作区配合

后续 AI 对话区将作为任务引导和分析入口，例如：

- 说明当前处理进度；
- 提示教师完成逐字稿校订；
- 解释某张证据的生成依据；
- 根据教师要求分析指定时间段；
- 帮助教师选择复盘重点；
- 根据教师反馈调整建议；
- 在异常发生时说明原因和可执行操作。

视频定位、逐字稿编辑、证据修改和报告确认仍在结构化工作区中完成。

未来的交互关系为：

AI 对话区

负责说明、引导、追问和触发任务

结构化工作区

负责视频核对、文本修改、证据复核和报告确认

2.4 关键状态

课堂处理流程包含以下主要状态：

```
created
queued
running
waiting_for_human
completed
failed
cancelled
```

前端将处理过程整理为教师更容易理解的阶段：

```
创建课堂
→ 上传素材
→ 处理音频和逐字稿
→ 校订原文
→ 生成和核对证据
→ 人工复核
→ 生成报告
```

系统包含两个重要的人工确认节点。

逐字稿校订节点

逐字稿完成处理和分段后，系统暂停后续分析，等待教师修改和确认。

教师确认后，系统才继续计算指标和生成教学证据。

证据复核节点

候选证据生成后，教师可以进行：

- 接受；
- 修改后接受；
- 驳回。

最终报告只读取已经接受或修改后接受的内容。

2.5 异常状态

当前系统可以识别部分处理异常，但完整日志、用户错误提示和自动重试能力尚未完成。

可能出现的异常包括：

- 视频或音频上传失败；
- ASR 调用失败；
- 外部模型返回格式不符合要求；
- 发言人身份识别不准确；
- 逐字稿与实际音频存在偏差；
- 后台处理任务中断；
- 候选证据缺少有效来源；
- 报告生成失败。

后续需要为每个处理步骤补充：

- 明确的错误状态；
- 面向教师的错误说明；
- 技术错误详情；
- 重试次数；
- 恢复入口；
- 失败步骤定位。

2.6 高价值交互设计

当前最有价值的交互设计是：

教师可以从逐字稿和教学证据直接跳转到对应视频时间点，并在修改逐字稿后，让后续分析使用修订后的文本。

这一设计将视频、逐字稿、课堂指标和教学证据连接在同一条时间轴上。

教师可以直接核对：

- 原话是否识别正确；
- 发言人是否判断合理；
- 等待时间是否符合真实课堂；
- 教学证据是否符合上下文；
- 建议是否适合当前课堂。

教师不需要在报告和长视频之间反复手工寻找片段，从而提高复盘效率、准确性和可信度。

三、实现与技术

3.1 技术设计原则

当前技术选型主要考虑三个目标。

快速完成真实产品闭环

项目由个人独立开发，因此优先选择能够降低前后端切换成本、减少重复代码并快速验证完整流程的技术。

保证课堂数据可追溯

课堂视频、逐字稿、时间点、指标、证据和教师复核结果需要形成稳定关联。因此，系统以结构化数据库和明确的数据契约为基础，而不是只保存模型生成的文本。

为后续阶段保留扩展空间

M2 将增加两轮课堂比较，M3 将增加多课程管理、Memory 和 RAG。当前架构需要支持增加新任务、新 Agent、新数据结构和新服务，而不需要重写全部系统。

3.2 整体工程结构

当前项目采用 pnpm Workspace 和 Turborepo 管理正式工程，主要包括：

```
apps/  
  web  
  api  
  worker  
  
packages/  
  database  
  providers  
  agents  
  metrics  
  guardrails  
  shared-types  
  api-contracts
```

采用 Monorepo 可以让前端、后端、后台任务和 AI 分析模块共享：

- TypeScript 类型；
- API 请求和响应结构；
- 课堂状态枚举；
- 证据字段；

- Workflow 步骤；
- 环境配置。

这可以减少个人开发中常见的前后端字段不一致问题，也便于未来将某个模块独立部署或替换。

3.3 前端技术

前端使用：

- Next.js；
- React；
- TypeScript。

前端主要实现：

- 课堂创建；
- 课堂类型选择；
- 视频上传；
- 上传进度展示；
- 视频播放；
- 逐字稿时间轴；
- 大段课堂记录编辑；
- 点击翻译；
- 证据查看与复核；
- 报告展示。

课堂复盘包含播放器、时间轴、编辑区、处理进度和证据卡等多个交互区域。React 适合将这些区域拆分成独立组件，Next.js 便于统一页面路由、构建和部署，TypeScript 则可以让前端直接复用后端定义的数据类型。

这一组合既提高个人开发效率，也为未来增加课堂库、跨课堂比较和多课程页面保留扩展空间。

3.4 后端 API

主业务接口使用：

- NestJS；
- TypeScript。

API 主要负责：

- 创建课堂记录；
- 生成上传凭证；
- 保存视频和音频元数据；
- 查询课堂处理状态；
- 保存逐字稿修改；
- 保存翻译结果；
- 保存证据复核结果；
- 触发报告生成。

NestJS 提供模块化的 Controller、Service 和依赖注入结构，适合将课堂、上传、逐字稿、证据和报告等业务分开管理。

M2 和 M3 将增加跨课堂比较、历史记录和课程管理。提前划分模块边界，可以降低后续扩展时的重构成本。

3.5 后台任务

后台任务使用独立 TypeScript Worker，负责：

- 获取待处理课堂；
- 检查视频和音频对象；
- 调用 ASR；
- 保存逐字稿；
- 运行逐字稿处理；
- 计算指标；
- 调用教学证据 Agent；
- 推进任务状态。

课堂视频处理、ASR 和模型分析可能持续较长时间，不适合放在普通 API 请求中同步执行。

独立 Worker 可以：

- 避免页面请求长时间等待；
- 持续保存任务状态；
- 为后续自动重试提供基础；
- 支持多个任务排队；
- 分别扩展 API 和后台计算资源。

Worker 与前后端共同使用 TypeScript，可以减少个人开发中的语言切换成本。未来如果部分智能任务需要 Python，可以通过独立 AI Runtime 接入。

3.6 数据库

系统使用 Supabase PostgreSQL 保存：

- 课堂基本信息；
- 课堂类型；
- 视频和音频对象信息；
- 处理状态；
- 带时间点的逐字稿；
- 大段课堂记录；
- 教师修改后的文本；
- 翻译结果；
- 课堂指标；
- 教学证据；
- 教师复核结果；
- 报告内容。

课堂复盘数据具有明显的关联关系：

课堂

- 视频
- 逐字稿
- 时间段
- 指标
- 证据
- 教师审核
- 报告

PostgreSQL 适合保存这类结构化、关联性强且需要追溯的数据。

Supabase 降低了个人开发中数据库部署和管理的成本，同时保留标准 PostgreSQL 能力。后续可以继续增加跨课堂关联、Memory、向量字段和 RAG 检索。

3.7 对象存储

系统使用 Cloudflare R2 保存：

- 原始课堂视频；
- ASR 使用的音频文件；
- 后续导出的报告文件。

浏览器通过签名地址将大文件直接上传到 R2。

课堂视频通常较大，不适合通过 API 服务中转，也不适合直接保存在数据库中。对象存储可以将大文件保存与业务计算分离：

- API 管理对象信息和状态；
- Worker 根据对象地址读取素材；
- 页面通过签名地址访问视频；
- 历史课堂可以重新分析；
- 后续可以独立扩展存储和计算资源。

3.8 外部模型服务

阿里云 DashScope ASR

ASR 用于将课堂音频转换为带时间点的逐字稿。

使用真实 ASR 服务而不是模拟逐字稿，是为了验证系统能否处理真实课堂中的口语、停顿、学生回答和录音误差。

大语言模型

大语言模型主要用于：

- 逐字稿语义整理；
- 课堂大段标题生成；
- 教学证据生成；
- 英文片段翻译；
- 教师建议表达；
- 不确定性说明。

外部模型通过 Provider 层调用。

Provider 层可以降低业务逻辑与具体供应商之间的耦合，使系统后续能够：

- 更换 ASR 或语言模型；
- 增加专业翻译服务；
- 为不同任务选择不同模型；
- 记录模型调用成本和耗时。

3.9 Transcript Agent

Transcript Agent 负责将 ASR 输出整理为教师可读、后续分析可用的课堂逐字稿。

当前采用一份标准化逐字稿派生两种用途的数据：

标准化逐字稿

- └ 展示投影
- └ 分析投影

展示投影主要用于教师阅读、时间轴展示、大段记录和人工修改；分析投影主要用于指标计算、互动关系分析和教学证据生成。

当前已实现

- 带时间点逐字稿；
- 大段课堂记录；
- 逐字稿编辑保存；
- 展示投影与分析投影；
- 课堂段落标题；
- 后续分析读取教师修改后的文本；
- 视频时间点与逐字稿关联。

Transcript Agent 设计要点

1. 统一事实源与双投影架构

系统不分别维护两份互不关联的逐字稿，而是从统一的标准化逐字稿派生展示投影和分析投影。

展示层可以优化阅读体验，分析层则保留时间点、发言人和其他分析信号。教师修改后的内容也能够继续进入下游分析。

2. 教学语义分段

逐字稿不只按照 ASR 片段或固定字数切分，而是结合教学主题、讲解、提问、回答、反馈、练习和总结等课堂活动形成大段记录。

每个大段生成具有教学含义的标题，帮助教师快速理解该时间段正在进行的教学活动。

3. 校订驱动的分析更新

教师校订不是只修改页面展示内容。系统使用校订后的文本重新进行指标和证据分析，避免继续基于错误的原始识别结果生成结论。

发言人或语音内容无法可靠判断时，系统保留不确定性，避免强制生成确定判断。

当前限制

当前仍不能稳定完成：

- 齐答和单个学生回答区分；
- 多人重叠语音处理；
- 完整句级版本记录；
- 系统化逐字稿质量评测；
- 高精度发言人角色判断。

3.10 Teaching Evidence Agent

Teaching Evidence Agent 根据以下输入生成候选教学证据：

- 教师校订后的逐字稿；
- 课堂类型；
- 基础课堂指标；
- 问题、等待、回答和反馈关系；
- 当前可分析能力；
- 不确定性信息。

当前已实现

- 至少三类候选证据卡；
- 证据关联时间点；
- 证据引用逐字稿原文；
- 问题、等待、回答和反馈链分析；
- 信息密度提示；
- 可分析能力矩阵；
- 不确定性提示；
- 教师接受；
- 教师修改后接受；
- 教师驳回；
- 报告只使用确认后的内容。

Teaching Evidence Agent 设计要点

1. 动态分析边界

系统通过可分析能力矩阵，根据线下课堂、直播网课和录播课程的实际数据条件限制分析范围。

例如，录播课程不会因为缺少学生回答而生成互动不足结论；直播课程缺少聊天或平台数据时，也不会推断学生在线行为。

2. 课堂互动链建模

系统将一次课堂互动组织为：

教师提问

→ 等待

→ 学生回答

→ 教师反馈或追问

分析结果不只包含提问数量，还能够显示问题后是否留出时间、是否出现回答，以及教师如何继续处理该回答。

3. 教师反馈转译

内部分析结果会被转换为教师可理解的课堂观察、教学意义、可执行建议和不确定性说明。建议尽量对应具体课堂片段，说明教师下一次可以如何调整，而不是只输出“增加互动”或“加强提问”等通用表达。

当前限制

以下设计尚未全部实现或系统测试：

- 五级学习检查；
- 笼统理解检查识别；
- 齐答和个体回答区分；
- 复习课、试卷讲评课和考试训练课的完整规则；
- 更细课堂结构；
- 大规模课堂片段评测。

3.11 内部指标与校验模块

Metrics Engine

基础指标模块用于计算：

- 语速；

- 连续讲授时间；
- 长停顿；
- 问题数量；
- 问题后等待时间；
- 回答与反馈关系；
- 信息密度相关提示。

当前已经完成：

- 基础指标计算；
- 问题—等待—回答—反馈链；
- 信息密度提示。

课堂节奏曲线和结尾加速提示尚未实现。

输出校验

系统对候选证据进行结构和来源检查，主要包括：

- 时间范围是否有效；
- 引用内容是否存在；
- 证据是否属于当前课堂；
- 分析维度是否适用于当前课堂类型；
- 是否包含不确定性说明；
- 是否能够回到原文和时间点。

将确定性计算和语义生成分开，可以避免语言模型重新计算语速、等待时间和问题数量，使数值结果更稳定，也便于后续独立测试。

3.12 部署方式

当前使用：

- Google Cloud Build；
- Google Cloud Run Web；
- Google Cloud Run API；
- Google Cloud Run Job Worker；
- Google Secret Manager；
- Cloudflare R2；
- Supabase PostgreSQL；
- 阿里云 DashScope。

Cloud Run 可以分别部署前端、API 和 Worker，不需要个人长期维护固定服务器。

Secret Manager 用于保存数据库、对象存储和模型服务密钥，避免将敏感信息写入代码。

这一部署方式能够满足 M1 的真实运行需求，并允许后续分别扩展页面、业务接口和后台分析资源。

四、完成范围与限制

4.1 已真实实现并运行

以下功能已经接入真实处理链路并能够运行：

- 创建课堂；
- 课堂类型选择；
- 视频上传到 Cloudflare R2；
- 视频对象和音频对象写入数据库；
- 上传进度真实显示；
- 独立音频通道；
- 真实 ASR；
- 带时间点逐字稿；
- 大段课堂记录；
- 逐字稿编辑保存；
- 点击翻译；
- 基础指标计算；
- 至少三类教学证据卡；
- 证据可回到时间点和原文；
- 证据接受；
- 证据修改后接受；
- 证据驳回；
- 报告只使用教师确认后的内容；
- 问题、等待、回答和反馈链分析；
- 信息密度提示；
- 可分析能力矩阵；
- 不确定性提示；
- 录播课程完整流程；
- 线下课堂视频完整流程。

当前产品在视频、外部服务和云端配置正常时，可以运行到课堂报告生成阶段。

4.2 部分实现

Transcript Agent

逐字稿生成、分段、标题、编辑和后续分析已经完成，但发言人识别仍受到录音质量和 ASR 说话人分离能力影响。

当前尚不能稳定区分：

- 多名学生同时发言；
- 全班齐声回答；
- 单个学生回答；
- 重叠语音中的不同发言人。

完整句级版本记录、复杂时间轴事件和逐字稿质量评估也尚未全部完成。

Teaching Evidence Agent

候选证据生成、时间定位、原文引用、证据修改、能力矩阵和不确定性提示已经实现。

部分规则仍处于设计或部分接入阶段，例如：

- 五级学习检查；
- 笼统理解检查；
- 齐答和个体回答区分；
- 更完整的复习课、试卷讲评课和考试训练分析；
- 更细课堂结构。

报告生成

当前已经能够根据教师确认后的证据生成报告。

由于个人开发时间和资源有限，独立 Report Agent 尚未完成。当前报告生成主要完成确认内容的整理，还未形成完整的：

- 独立输入输出结构；
- 报告版本记录；
- 报告生成过程校验；
- 多格式导出；
- 报告生成质量评测。

4.3 尚未实现

M1 尚未实现

- AI 对话区；
- 基础日志；
- 面向用户的完整错误提示；
- 自动重试和步骤恢复；
- 独立 Report Agent；
- 完整 Observability；
- 完整 Evaluation；
- 课堂节奏曲线；
- 结尾加速提示；
- 齐答与个体回答区分；
- 笼统理解检查；
- 直播聊天记录 CSV 导入；
- 课件页关联；
- 课堂画面截图作为证据；
- 更细课堂结构；
- PDF 导出。

Observability 后续内容

后续需要记录：

- Workflow 当前步骤；
- 失败步骤和错误原因；
- 模型名称；
- Prompt 版本；
- 模型调用耗时；
- 输入和输出 Token；
- 重试次数；
- 输出结构校验结果；
- 生成的事件和证据数量；
- 外部模型调用成本。

Evaluation 后续内容

后续需要建立离线评测集和回归测试，覆盖：

- 正常中文讲授；

- 发言人识别错误；
- 教师自问自答；
- 全班齐答；
- 个体学生回答；
- 提问后长等待；
- 课堂管理语言；
- 中英混合；
- 学生声音较弱；
- 直播技术中断；
- 录播课程没有学生互动；
- 试卷讲评课；
- 复习课。

4.4 模拟展示说明

本报告未将静态界面、示例数据、Prompt 示例和文档中的预期输出计入已实现功能。

尚未接入真实流程的 Agent 规则、分析能力和交互设计均列入部分实现或尚未实现，不作为当前完成结果。

4.5 当前限制原因

尚未完成的功能主要受到以下因素影响：

- 项目由个人独立完成；
- 开发周期有限；
- 缺少专门的前端、后端、算法和测试人员；
- 云端模型和长视频处理存在调用成本；
- 可用于测试的真实课堂视频数量有限；
- 缺少经过人工标注的课堂数据集；
- 说话人识别受麦克风位置、噪声和多人重叠语音影响；
- 多个云服务之间的配置和调试工作量较大；
- Observability、Evaluation 和系统测试需要额外工程投入。

这些限制主要影响功能完整度、稳定性和测试覆盖，不改变当前主流程已经能够真实运行的结果。

五、M1 以后开发计划

5.1 第一阶段：完成 M1 剩余能力

AI 对话

实现 AI 对话区，使教师可以：

- 询问当前处理状态；
- 追问某张证据的来源；
- 要求分析指定视频片段；
- 选择复盘重点；
- 要求重新生成或调整建议；
- 在失败时获得明确处理提示。

Report Agent

将当前报告整理逻辑升级为独立 Report Agent。

Report Agent 读取：

- 课堂基本信息；
- 已确认指标；
- 已接受或修改后接受的证据；
- 教师确认后的建议；
- 分析限制和不确定性。

Report Agent 不重新分析原始逐字稿，也不增加未经教师确认的新判断。

Observability

建立统一运行记录，追踪：

- 每个处理步骤；
- 外部服务调用；
- 模型和 Prompt 版本；
- Token 和调用成本；
- 失败原因；
- 重试过程；
- 证据校验结果。

Evaluation

建立小规模标准课堂片段测试集，持续验证：

- 逐字稿时间点；
- 发言人识别；
- 提问和回答关系；
- 教学证据引用；
- 不确定性提示；
- 分析边界；
- 报告内容一致性。

其他 M1 功能

继续实现：

- 课堂节奏曲线；
- 结尾加速提示；
- 齐答与个体回答区分；
- 笼统理解检查；
- 更细课堂结构；
- PDF 导出；
- 直播聊天记录 CSV 导入；
- 课件页关联；
- 课堂画面截图作为补充证据。

5.2 M2：比较第二轮课堂是否改进

M2 的核心任务是将教师确认后的建议转化为下一次课堂的具体改进行动。

本轮发现

→ 教师确认的问题

→ 下一轮准备采取的动作

→ 需要观察的课堂行为

→ 下一轮视频中的对应证据

当同一门课再次录制后，系统对比两次课堂，分析：

- 教师是否实施上一次确认的改进；
- 提问后等待时间是否变化；
- 是否增加学生表达机会；
- 教师反馈方式是否变化；

- 信息密度和课堂结构是否变化；
- 改动是否出现在相似教学环节；
- 当前证据是否足以支持改进判断。

M2 不只比较两次课堂的总指标，而是尽量寻找相似教学内容和教学环节进行比较。

M2 Memory

M2 需要保存：

- 教师确认的问题；
- 教师接受的建议；
- 下一轮准备实施的动作；
- 需要继续观察的指标；
- 两轮课堂之间的对应关系；
- 教师对第二轮结果的确认。

M2 RAG

M2 可以加入轻量 RAG，用于：

- 检索上一轮课堂中的相似教学情境；
- 查找与某项改进建议相关的历史证据；
- 匹配两轮课堂中的相似提问、练习和总结片段；
- 为跨课堂比较提供来源和时间点。

5.3 M3：多门课程管理和汇总分析

M3 在 M1 和 M2 稳定后扩展到多门课程管理。

主要功能包括：

- 多门课程列表；
- 同一课程的多次课堂记录；
- 不同课堂之间的对比；
- 教师长期改进趋势；
- 课程级汇总报告；
- 多维度筛选和检索。

M3 第一版不需要直接建设覆盖全校的综合管理平台，应继续控制产品范围。

M3 Memory

M3 可以形成长期教学改进记忆，包括：

- 多次课堂中重复出现的问题；
- 已经完成的改进；
- 长期未改善的问题；
- 教师偏好的建议类型；
- 不同课程中的教学策略；
- 已确认的课堂亮点；
- 不同阶段的复盘目标。

M3 RAG

M3 的 RAG 可以扩大到：

- 多门课程；
- 多轮课堂逐字稿；
- 历史证据；
- 历史报告；
- 教师确认的建议；
- 课件和教学资料；
- 教学规范和评价标准。

检索结果仍需保留来源、时间点和课程信息。

5.4 可选扩展：脑图和知识图谱

在主要课堂复盘和跨课堂比较稳定后，可以根据课件和逐字稿生成：

- 课堂知识脑图；
- 知识点关系图；
- 教学环节图；
- 问题与回答关系图；
- 错误类型与知识点关系图。

图中的每个知识点和关系都需要：

- 能够跳转到课堂原文；
- 能够定位到视频时间点；
- 标明信息来源；
- 允许教师修改；
- 经过教师确认后保存。

六、测试记录

6.1 当前测试范围

目前完成了：

- 一次录播课程完整流程测试；
- 一次线下课堂视频完整流程测试。

两次测试均覆盖：

创建课堂

→ 上传视频

→ 生成逐字稿

→ 编辑逐字稿

→ 点击翻译

→ 计算基础指标

→ 生成教学证据

→ 复核证据

→ 生成报告

两种课堂类型均能够运行到报告阶段。

目前尚未进行系统化测试，包括：

- 大规模视频测试；
- 多种文件格式兼容测试；
- 并发任务测试；
- 长时间稳定性测试；
- 不同浏览器测试；
- 完整异常恢复测试；
- Agent 离线评测；
- 自动回归测试；
- 压力测试；
- 模型成本测试。

因此，当前测试结果能够证明主流程可以运行，但不能证明系统已经达到大规模生产环境的稳定性要求。

6.2 录播课完整流程测试

录播课测试重点验证：

- 视频上传；
- ASR 转写；
- 带时间点逐字稿；
- 大段课堂记录；
- 逐字稿编辑；
- 英文片段翻译；
- 信息密度提示；
- 录播课程分析边界；
- 证据复核；
- 报告生成。

测试中重点检查系统是否错误生成学生参与、学生回答或等待学生回答相关结论。

测试结果表明，系统能够按照录播课程的可分析能力限制分析范围，主要生成课程结构、语言表达、信息密度和教学内容相关证据。

6.3 线下课堂视频完整流程测试

线下课堂测试重点验证：

- 师生语音逐字稿；
- 教师提问识别；
- 问题后等待时间；
- 学生回答；
- 教师反馈；
- 视频时间点跳转；
- 教师修改逐字稿；
- 候选证据生成；
- 证据修改和确认；
- 报告生成。

测试结果表明，问题—等待—回答—反馈链可以进入后续分析，但发言人识别准确性仍是当前主要限制。

6.4 常见失败场景：发言人识别不准确

失败现象

在线下课堂视频中，ASR 的说话人分离结果可能出现：

- 将教师语音识别为学生；
- 将学生回答识别为教师；
- 将同一个学生拆分为多个发言人；
- 将多名学生同时发言识别为单一发言人；
- 无法判断某段声音是齐答还是单个学生回答。

产生原因

发言人识别受到以下因素影响：

- 教室录音设备距离教师较近；
- 学生声音较小；
- 教室内存在回声；
- 多名学生同时发言；
- 教师和学生声音发生重叠；
- ASR 说话人分离能力有限；
- 视频中没有额外身份数据。

测试中发现的问题

发言人识别不准确会进一步影响：

- 问题和回答的对应关系；
- 学生回答次数；
- 教师自问自答判断；
- 等待时间计算；
- 教师反馈对象；
- 教学证据的置信度。

已进行的修改

当前分析增加了以下限制：

- 发言人标签不作为唯一判断依据；
- 结合前后文、问题关系和语言行为辅助判断；
- 无法可靠判断时增加不确定性提示；
- 低置信度片段不生成强结论；
- 教学证据保留原文和时间点，便于教师核对；
- 教师可以通过修改逐字稿减少后续分析偏差。

仍需完成

后续仍需增加：

- 发言人角色人工修改；
- 齐答和单个学生回答区分；
- 多人重叠语音处理；
- 发言人识别置信度展示；
- 修改发言人后自动重新分析；
- 相关离线测试片段。

七、AI 协作

7.1 产品运行中的智能能力

产品运行中的智能能力主要包括：

- Transcript Agent；
- Teaching Evidence Agent；
- 外部 ASR；
- 外部大语言模型。

Transcript Agent 负责逐字稿整理，Teaching Evidence Agent 负责课堂证据生成。ASR 提供语音识别结果，大语言模型提供语言理解、翻译和文本生成能力。

指标计算、流程状态和结构校验由程序模块完成，使语义生成与确定性处理保持分工。

7.2 AI 参与的开发工作

AI 在开发过程中参与了以下工作：

调研

- 调研课堂复盘产品流程；
- 比较课堂视频分析和普通文本总结的差异；
- 调研 ASR、对象存储和后台任务方案；
- 调研 Agent、Workflow、RAG 和 Memory 的适用场景。

方案设计

- 辅助整理 M1、M2 和 M3 范围；

- 辅助划分前端、API、Worker、数据层和模型调用层；
- 辅助设计课堂处理 Workflow；
- 辅助设计逐字稿展示和分析结构；
- 辅助整理证据数据结构；
- 辅助设计不同课堂类型的能力边界。

代码开发

- 生成部分 TypeScript 类型；
- 生成部分接口和数据结构草稿；
- 辅助编写页面、API 和 Worker 代码；
- 辅助编写第三方服务适配代码；
- 辅助完成重复性修改和目录重构。

调试和测试

- 分析 TypeScript 编译错误；
- 检查接口字段一致性；
- 检查环境变量和 Secret 配置；
- 辅助定位模型调用错误；
- 辅助生成测试场景；
- 辅助整理发言人识别失败案例；
- 辅助检查分析结果是否超出素材范围。

7.3 人工判断和修改

AI 提供的代码、规则和文本均经过人工判断和修改。

由我最终确定的内容包括：

- 项目实际完成范围；
- M1、M2 和 M3 的阶段边界；
- 技术选型和模块边界；
- Transcript Agent 的双投影结构；
- Teaching Evidence Agent 的能力矩阵和输出设计；
- 教师修改逐字稿后如何影响后续分析；
- 不同课堂类型的分析范围；
- 课堂证据是否符合真实视频内容；
- 建议表达是否适合教师阅读；

- 代码是否能够编译和运行；
- 云端服务是否正确连接；
- 测试结果是否可以作为完成功能的依据。

实际功能以人工修改后的代码和真实流程测试结果为准。

八、个人完成内容

8.1 项目分工

本项目由我个人独立完成，没有团队成员参与分工。

我负责：

- 项目范围确定；
- 用户需求分析；
- 产品流程设计；
- 前端开发；
- 后端 API 开发；
- Worker 开发；
- 数据库设计；
- 对象存储接入；
- ASR 和语言模型接入；
- Transcript Agent 设计与实现；
- Teaching Evidence Agent 设计与实现；
- 指标计算；
- 证据复核流程；
- 报告生成；
- 云端部署；
- 流程测试；
- 技术文档编写。

8.2 个人主要设计与实现

标准化逐字稿与双投影结构

我设计了统一的标准化逐字稿，并从中派生面向教师的展示投影和面向分析模块的分析投影。

该设计使教师看到的文本与后续分析使用的数据保持一致，也为保留时间点、发言人和课堂信号提供了结构基础。

语义大段课堂记录

我将碎片化 ASR 结果按照课堂语义和教学活动整理为大段记录，并生成具有教学含义的标题。

这一设计降低了教师逐句阅读和修改的操作负担。

教师校订参与后续分析

教师修改逐字稿后，系统使用修订内容重新进行指标和证据分析，而不是继续使用原始识别结果。

可分析能力矩阵

我为线下课堂、直播网课和录播课程设置不同的分析范围，使系统根据实际可获得的数据决定可以生成哪些分析结果。

问题—等待—回答—反馈链

我将课堂互动组织为连续关系，而不是只统计提问数量，使证据能够反映一次完整互动过程。

教师反馈转译

我将内部分析结果转换为课堂观察、教学意义、可执行建议和不确定性说明，使输出更适合教师真实复盘。

面向后续扩展的模块化设计

页面、API、后台任务、数据访问、外部服务、智能分析、指标计算和输出校验被拆分为不同模块。

这一结构可以支持 M2 增加 Memory、RAG 和两轮课堂比较，也可以支持 M3 增加多课程管理。

8.3 遇到的主要问题

开发过程中主要遇到以下问题：

- 大视频文件不适合通过普通 API 上传；

- 浏览器、数据库和对象存储需要保持上传状态一致；
- ASR 输出较碎，不能直接用于课堂复盘；
- 发言人识别容易受到教室噪声和重叠语音影响；
- 教师修改逐字稿后，已有分析需要重新处理；
- 语言模型可能生成缺少来源或超出素材范围的判断；
- 不同课堂类型不能使用完全相同的分析规则；
- Web、API 和 Worker 需要保持同步部署；
- 多个外部云服务增加了配置和调试成本；
- 个人开发时间不足以完成全部计划功能；
- 缺少系统测试所需的真实视频和人工标注数据。

8.4 最终完成结果

目前项目已经完成一条能够真实运行的课堂复盘主流程：

真实课堂视频

- 视频和音频上传
- 真实 ASR
- 带时间点逐字稿
- 教师校订
- 点击翻译
- 基础指标
- 问题、等待、回答、反馈分析
- 候选教学证据
- 教师接受、修改或驳回
- 生成确认后的课堂报告

我已经使用：

- 一段录播课程视频；
- 一段线下课堂视频；

分别完成过一次完整流程测试。

当前系统能够证明核心流程可以运行，但由于个人开发时间、开发资源和测试数据有限，尚未完成系统化测试、AI 对话、独立 Report Agent、Observability、Evaluation、PDF 导出和部分精细课堂分析功能。

后续开发将先补齐 M1 的稳定性、报告生成和评测能力，再推进 M2 的两轮课堂改进比较，以及 M3 的多课程管理、Memory、RAG 和长期教学复盘。

Part II: English Version

1. Problem and Target Users

1.1 Target Users

The system is designed primarily for classroom teachers who need to review and reflect on completed lessons. The current M1 scope supports:

- Offline classroom recordings;
- Recorded live online lessons;
- Pre-recorded online courses.

M1 focuses on helping an individual teacher review one lesson. It does not currently target school administrators, regional teaching-research departments, or large-scale organizational management.

1.2 Real-World Scenario

After completing a lesson, teachers often need to review the video and locate specific moments involving questions, wait time, student responses, teacher feedback, continuous lecturing, information density, and lesson structure.

The traditional reflection process presents several difficulties:

1. Full-length lesson videos are time-consuming to review manually;
2. Standard speech recognition produces fragmented transcripts that are difficult to use directly;
3. Transcripts, video moments, and analysis results are often disconnected;
4. General-purpose language models may overinterpret limited classroom evidence;
5. Offline, live online, and recorded lessons provide different observable information;
6. Teachers need specific and actionable suggestions rather than broad evaluations.

1.3 Intended Outcome and Success Criteria

The goal of M1 is to deliver a complete and operational reflection workflow for one real lesson:

Upload a classroom video

- Generate a timestamped transcript
- Allow the teacher to edit the transcript
- Calculate classroom metrics
- Generate candidate teaching evidence
- Let the teacher accept, edit, or reject the evidence
- Generate a report from confirmed content

The M1 success criteria are:

- The video can be uploaded and stored;
- The transcript is generated through a real ASR service;
- Transcript content can be linked to video timestamps;
- Teachers can correct transcription errors;
- Revised transcript content is used in downstream analysis;
- Evidence is linked to original text and video locations;
- Teachers can accept, edit and accept, or reject evidence;
- The final report uses only teacher-confirmed content.

1.4 Why the Scope Is Limited

The project was completed independently by one developer, with limited development time, cloud-service budget, test materials, and engineering resources.

Attempting to build multi-user management, school administration, slide analysis, visual recognition, cross-lesson comparison, knowledge graphs, and long-term teacher profiles in the first version would have made it difficult to complete the core classroom reflection workflow.

M1 therefore focuses on processing one real classroom video and validating whether:

- Video materials can enter the system reliably;
- Teachers can read, locate, and edit transcripts;
- Classroom metrics and analysis can be linked to real video segments;
- Confirmed findings can be turned into a usable report.

This scope addresses the immediate problems of time-consuming video review, difficult evidence location, and low trust in unsupported analysis.

2. Solution and Interaction Design

2.1 Core Workflow

The currently implemented workflow is:

```
Create a lesson
→ Select the lesson format
→ Upload the classroom video
→ Store video and audio object records
→ Start background processing
→ Call a real ASR service
→ Store a timestamped transcript
→ Process the transcript with the Transcript Agent
→ Build larger semantic lesson sections
→ Let the teacher edit and save the transcript
→ Confirm transcript review
→ Calculate classroom metrics
→ Analyse the question–wait–response–feedback chain
→ Generate candidate evidence with the Teaching Evidence Agent
→ Accept, edit and accept, or reject evidence
→ Generate a report from confirmed content
```

The current product relies primarily on a structured workspace.

The AI conversation area has not yet been implemented. Teachers cannot currently control the workflow through continuous natural-language conversation, ask why specific evidence was generated, or request conversational reanalysis of a selected segment.

2.2 Current Workspace

The current workspace supports:

- Lesson creation;
- Lesson-format selection;
- Video upload with real progress display;
- Video playback;
- Timestamped transcript viewing;
- Editing and saving semantic lesson sections;
- On-demand translation;
- Processing-status viewing;

- Classroom metric viewing;
- Candidate evidence review;
- Navigation from evidence to video timestamps;
- Evidence acceptance, editing, and rejection;
- Final report viewing.

The main user experience is built around the video player, transcript, metric cards, and evidence cards rather than a chat-only interface.

2.3 Planned Relationship Between Conversation and Workspace

The future AI conversation area will support task guidance and analysis control, including:

- Explaining the current processing stage;
- Prompting the teacher to review the transcript;
- Explaining why a piece of evidence was generated;
- Analysing a teacher-selected time range;
- Helping the teacher choose reflection priorities;
- Adjusting suggestions based on teacher feedback;
- Explaining errors and available recovery actions.

Video review, transcript editing, evidence modification, and report confirmation will remain in the structured workspace.

The planned interaction model is:

AI Conversation Area

Provides explanations, guidance, follow-up questions, and task triggers

Structured Workspace

Supports video verification, transcript editing, evidence review, and report confirmation

2.4 Key States

The processing workflow uses the following main states:

```
created
queued
running
waiting_for_human
completed
failed
cancelled
```

The interface presents these states as teacher-friendly stages:

```
Create lesson
→ Upload materials
→ Process audio and transcript
→ Review transcript
→ Generate and verify evidence
→ Human review
→ Generate report
```

There are two key human confirmation points.

Transcript Review

After transcript processing and segmentation, the workflow pauses until the teacher edits and confirms the transcript.

Only then does the system continue with metric calculation and evidence generation.

Evidence Review

After candidate evidence is generated, the teacher can:

- Accept it;
- Edit and accept it;
- Reject it.

The final report reads only accepted or edited-and-accepted evidence.

2.5 Error States

The system can currently detect some processing errors, but complete logging, user-facing error messages, and automatic retry mechanisms have not yet been implemented.

Possible errors include:

- Video or audio upload failure;
- ASR service failure;
- Invalid model output structure;
- Incorrect speaker identification;
- Transcript mismatch with the original audio;
- Interrupted background processing;
- Evidence without a valid source;
- Report-generation failure.

Future development needs to add:

- Clear error states;
- Teacher-facing explanations;
- Technical error details;
- Retry counts;
- Recovery actions;
- Failed-step identification.

2.6 High-Value Interaction

The most valuable interaction is the ability to move directly from transcript text or an evidence card to the corresponding video timestamp.

After a transcript correction, downstream analysis uses the revised text.

This connects the video, transcript, metrics, and evidence through one shared timeline. Teachers can verify whether:

- The original statement was transcribed correctly;
- The speaker was identified reasonably;
- The measured wait time reflects the real classroom;
- The evidence matches the surrounding context;
- The suggestion is appropriate for the lesson.

This reduces manual searching and improves reflection efficiency, accuracy, and trust.

3. Implementation and Technology

3.1 Technical Design Principles

The technology choices were guided by three goals.

Deliver a Real Product Workflow Quickly

Because the project was developed independently, the technology stack needed to reduce context switching, minimise duplicate code, and support rapid validation of an end-to-end workflow.

Maintain Traceable Classroom Data

Videos, transcripts, timestamps, metrics, evidence, and teacher decisions must remain connected. The system therefore relies on structured data and shared contracts rather than storing only model-generated text.

Preserve Future Extensibility

M2 will add comparison between two lesson rounds, while M3 will add multi-course management, Memory, and RAG. The architecture must support new tasks, Agents, data structures, and services without rebuilding the M1 pipeline.

3.2 Overall Project Structure

The project uses pnpm Workspace and Turborepo:

```
apps/  
  web  
  api  
  worker  
  
packages/  
  database  
  providers  
  agents  
  metrics  
  guardrails  
  shared-types  
  api-contracts
```

The Monorepo allows the frontend, API, Worker, and analysis modules to share:

- TypeScript types;
- API request and response schemas;
- Lesson-status enums;
- Evidence fields;
- Workflow steps;
- Configuration rules.

This reduces interface inconsistencies and makes it easier to deploy or replace individual modules later.

3.3 Frontend

The frontend uses:

- Next.js;
- React;
- TypeScript.

It supports:

- Lesson creation;
- Lesson-format selection;
- Video upload;
- Upload progress;
- Video playback;

- Transcript timelines;
- Editing of semantic lesson sections;
- On-demand translation;
- Evidence review;
- Report display.

Classroom reflection requires several connected interaction areas, including a player, timeline, editor, progress view, and evidence cards.

React supports component-based development, Next.js simplifies routing, building, and deployment, and TypeScript allows the frontend to reuse backend data definitions.

This combination supports efficient individual development while leaving room for lesson libraries, cross-lesson comparison, and multi-course pages.

3.4 Backend API

The main API uses:

- NestJS;
- TypeScript.

It is responsible for:

- Creating lesson records;
- Generating upload credentials;
- Storing video and audio metadata;
- Querying processing status;
- Saving transcript edits;
- Saving translations;
- Saving evidence-review decisions;
- Triggering report generation.

NestJS provides modular Controllers, Services, and dependency injection. This makes it easier to separate lesson, upload, transcript, evidence, and report responsibilities.

The modular structure also supports future M2 and M3 requirements such as historical lesson records and cross-lesson comparison.

3.5 Background Worker

A separate TypeScript Worker handles:

- Retrieving queued lessons;
- Checking video and audio objects;
- Calling ASR;
- Saving transcripts;
- Running transcript processing;
- Calculating metrics;
- Calling the Teaching Evidence Agent;
- Updating workflow states.

Video processing, ASR, and model analysis may take a long time and should not run synchronously inside standard API requests.

A separate Worker allows the system to:

- Avoid long-running page requests;
- Save task progress continuously;
- Add retry support later;
- Queue multiple tasks;
- Scale API and background resources independently.

Using TypeScript across the application reduces language-switching overhead. A separate Python AI Runtime can be added later if required.

3.6 Database

The system uses Supabase PostgreSQL to store:

- Lesson information;
- Lesson formats;
- Video and audio metadata;
- Processing states;
- Timestamped transcripts;
- Semantic lesson sections;
- Teacher-edited text;
- Translation results;
- Classroom metrics;
- Teaching evidence;
- Teacher-review decisions;
- Reports.

The data has a clear relational structure:

```
Lesson
→ Video
→ Transcript
→ Time segment
→ Metric
→ Evidence
→ Teacher review
→ Report
```

PostgreSQL is suitable for structured, relational, and traceable data.

Supabase reduces database deployment and management overhead while preserving standard PostgreSQL capabilities. Future stages can add cross-lesson relations, Memory, vector fields, and RAG retrieval.

3.7 Object Storage

Cloudflare R2 stores:

- Original classroom videos;
- Audio files used for ASR;
- Future exported reports.

The browser uploads large files directly through signed URLs.

Classroom videos are too large to route through the standard API or store inside the database. Object storage separates large-file persistence from business processing:

- The API manages metadata and status;
- The Worker reads materials from object references;
- The frontend accesses videos through signed URLs;
- Historical lessons can be reprocessed;
- Storage and compute resources can scale separately.

3.8 External Model Services

Alibaba Cloud DashScope ASR

ASR converts classroom audio into timestamped transcripts.

Using a real ASR service instead of simulated transcripts validates the system against real classroom speech, pauses, student responses, and recording errors.

Large Language Model

The language model is used for:

- Semantic transcript processing;
- Lesson-section title generation;
- Teaching evidence generation;
- Translation;
- Teacher-facing suggestion wording;
- Uncertainty descriptions.

External models are accessed through a Provider layer.

The Provider layer reduces dependency on a single model vendor and supports:

- Replacing ASR or language models;
- Adding professional translation services;
- Selecting different models for different tasks;
- Recording model latency and cost.

3.9 Transcript Agent

The Transcript Agent converts raw ASR output into a transcript that is useful for both teachers and downstream analysis.

It uses one standardised transcript to derive two projections:

```
Canonical Transcript
├─ Display Projection
└─ Analysis Projection
```

The display projection supports reading, timeline navigation, semantic sections, and editing. The analysis projection supports metrics, interaction analysis, and evidence generation.

Implemented Capabilities

- Timestamped transcripts;

- Larger semantic lesson sections;
- Transcript editing and saving;
- Display and analysis projections;
- Lesson-section titles;
- Downstream analysis based on teacher edits;
- Transcript-to-video timestamp links.

Transcript Agent Design Points

1. Canonical Source and Dual Projections

The system does not maintain two unrelated transcripts. Both the teacher-facing view and the analysis view are derived from one standardised source.

This keeps teacher edits, timestamps, and downstream analysis consistent.

2. Pedagogical Semantic Segmentation

Transcript sections are created based on teaching activities such as explanation, questioning, response, feedback, practice, and summary, rather than only ASR boundaries or fixed character counts.

Each section receives a meaningful title that helps teachers understand the activity taking place.

3. Revision-Driven Analysis

Teacher corrections are used in downstream metric and evidence generation.

When speech or speaker identity cannot be determined reliably, the system preserves uncertainty rather than forcing a definite interpretation.

Current Limitations

The current implementation cannot yet reliably support:

- Choral versus individual response identification;
- Overlapping multi-speaker processing;
- Complete sentence-level version history;
- Systematic transcript quality evaluation;
- High-accuracy speaker-role classification.

3.10 Teaching Evidence Agent

The Teaching Evidence Agent generates candidate evidence from:

- Teacher-reviewed transcripts;
- Lesson format;
- Classroom metrics;
- Question–wait–response–feedback relationships;
- Current analysis capabilities;
- Uncertainty information.

Implemented Capabilities

- At least three categories of evidence cards;
- Timestamp-linked evidence;
- Transcript quotations;
- Question–wait–response–feedback analysis;
- Information-density prompts;
- Capability Matrix;
- Uncertainty notes;
- Evidence acceptance;
- Evidence editing and acceptance;
- Evidence rejection;
- Reports based only on confirmed evidence.

Teaching Evidence Agent Design Points

1. Dynamic Analysis Boundaries

A Capability Matrix determines which types of analysis are valid for offline, live online, and recorded lessons.

For example, recorded lessons are not criticised for missing student responses, and live lessons without platform data do not generate claims about attendance or online participation.

2. Classroom Interaction Chain

The system models classroom interaction as:

Teacher question

→ Wait time

→ Student response

→ Teacher feedback or follow-up

This provides more useful information than question counts alone and allows evidence to represent a complete interaction sequence.

3. Teacher-Facing Translation

Internal analytical results are converted into teacher-readable observations, teaching implications, actionable suggestions, and uncertainty notes.

Suggestions are linked to specific classroom moments rather than relying on generic statements such as “increase interaction.”

Current Limitations

The following areas remain incomplete or insufficiently tested:

- Five-level learning checks;
- Generic comprehension-check detection;
- Choral versus individual response identification;
- Full support for review lessons, test-paper review, and exam training;
- More detailed classroom structure;
- Large-scale classroom-segment evaluation.

3.11 Metrics and Validation

Metrics Engine

The Metrics Engine calculates:

- Speech rate;
- Continuous lecturing duration;
- Long pauses;
- Number of questions;
- Wait time after questions;
- Response and feedback relationships;
- Information-density prompts.

Currently implemented:

- Basic metrics;
- Question–wait–response–feedback chains;
- Information-density prompts.

Classroom rhythm curves and end-of-lesson acceleration detection have not yet been implemented.

Output Validation

Candidate evidence is checked for:

- Valid time ranges;
- Existing source text;
- Correct lesson ownership;
- Compatibility with the lesson format;
- Presence of uncertainty notes;
- Traceability to transcript text and timestamps.

Separating deterministic calculation from semantic generation improves numerical reliability and supports independent testing.

3.12 Deployment

The current deployment uses:

- Google Cloud Build;
- Google Cloud Run Web;
- Google Cloud Run API;
- Google Cloud Run Job Worker;
- Google Secret Manager;
- Cloudflare R2;
- Supabase PostgreSQL;
- Alibaba Cloud DashScope.

Cloud Run allows the frontend, API, and Worker to be deployed separately without maintaining permanent servers.

Secret Manager protects database, storage, and model-service credentials.

This deployment approach supports the current M1 workflow while leaving room to scale interface, API, and background processing resources independently.

4. Completion Scope and Limitations

4.1 Fully Implemented and Running

The following capabilities are connected to the real workflow and operational:

- Lesson creation;
- Lesson-format selection;
- Video upload to Cloudflare R2;
- Video and audio object records stored in the database;
- Real upload progress;
- Independent audio channel;
- Real ASR;
- Timestamped transcripts;
- Larger semantic lesson sections;
- Transcript editing and saving;
- On-demand translation;
- Basic metric calculation;
- At least three categories of evidence cards;
- Evidence linked to timestamps and original text;
- Evidence acceptance;
- Evidence editing and acceptance;
- Evidence rejection;
- Reports based only on confirmed content;
- Question–wait–response–feedback analysis;
- Information-density prompts;
- Capability Matrix;
- Uncertainty notes;
- Complete recorded-lesson workflow;
- Complete offline-classroom workflow.

When the video, external services, and cloud configuration are available, the product can run through report generation.

4.2 Partially Implemented

Transcript Agent

Transcript generation, segmentation, titles, editing, and downstream use are implemented, but speaker identification remains sensitive to recording quality and ASR diarisation.

The system cannot yet reliably distinguish:

- Multiple students speaking together;
- Choral responses;
- Individual student responses;
- Different speakers in overlapping audio.

Complete sentence-level versioning, complex timeline events, and transcript quality evaluation are also incomplete.

Teaching Evidence Agent

Evidence generation, timestamp links, quotations, editing, the Capability Matrix, and uncertainty notes are implemented.

The following areas remain at the design or partial-integration stage:

- Five-level learning checks;
- Generic comprehension-check detection;
- Choral versus individual responses;
- Full review-lesson, test-paper review, and exam-training support;
- More detailed classroom structure.

Report Generation

The current system can generate reports from confirmed evidence.

Due to limited development time and resources, an independent Report Agent has not yet been completed. The current report workflow does not yet include:

- A complete independent input/output contract;
- Report versioning;
- Full report-generation validation;
- Multiple export formats;
- Report-quality evaluation.

4.3 Not Yet Implemented

Remaining M1 Capabilities

- AI conversation area;
- Basic logging;
- Complete user-facing error messages;
- Automatic retries and step recovery;
- Independent Report Agent;
- Full Observability;
- Full Evaluation;
- Classroom rhythm curve;
- End-of-lesson acceleration detection;
- Choral versus individual response identification;
- Generic comprehension-check detection;
- Live-chat CSV import;
- Slide-page association;
- Classroom screenshots as evidence;
- More detailed classroom structure;
- PDF export.

Planned Observability

Future Observability should record:

- Current Workflow step;
- Failed step and error reason;
- Model name;
- Prompt version;
- Model latency;
- Input and output tokens;
- Retry count;
- Output-schema validation;
- Number of generated events and evidence cards;
- External model cost.

Planned Evaluation

Future evaluation should cover:

- Standard Chinese teaching;
- Speaker-identification errors;
- Teacher self-answering;
- Choral response;
- Individual response;
- Long wait after a question;
- Classroom-management language;
- Mixed Chinese and English;
- Weak student audio;
- Live-stream technical interruptions;
- Recorded lessons without student interaction;
- Test-paper review lessons;
- Review lessons.

4.4 Simulated Content

Static interfaces, example data, Prompt examples, and expected outputs from design documents are not counted as implemented functions.

Agent rules, analytical capabilities, and interaction concepts that are not connected to the real workflow are classified as partial or unimplemented.

4.5 Reasons for Current Limitations

The main constraints are:

- The project was completed independently;
- Development time was limited;
- There were no dedicated frontend, backend, algorithm, or testing team members;
- Cloud models and long-video processing have operational costs;
- Only a limited number of real classroom videos were available;
- No manually labelled classroom dataset was available;
- Speaker identification is affected by microphone placement, noise, and overlapping speech;
- Multi-cloud configuration and debugging required significant effort;
- Observability, Evaluation, and systematic testing require additional engineering work.

These limitations affect completeness, stability, and test coverage, but do not change the fact that the core M1 workflow is operational.

5. Development Plan After M1

5.1 Phase One: Complete the Remaining M1 Capabilities

AI Conversation

Implement an AI conversation area that allows teachers to:

- Ask about processing status;
- Ask for the source of an evidence card;
- Request analysis of a selected video segment;
- Select reflection priorities;
- Request revised suggestions;
- Receive clear guidance after failures.

Report Agent

Upgrade the current report logic into an independent Report Agent.

The Report Agent will read:

- Lesson information;
- Confirmed metrics;
- Accepted or edited-and-accepted evidence;
- Teacher-confirmed suggestions;
- Limitations and uncertainty.

It will not reanalyse the raw transcript or add unreviewed conclusions.

Observability

Build unified runtime records covering:

- Workflow steps;
- External-service calls;
- Model and Prompt versions;
- Token usage and cost;

- Failure reasons;
- Retry processes;
- Evidence-validation results.

Evaluation

Create a small standardised classroom test set to evaluate:

- Transcript timestamps;
- Speaker identification;
- Question-response relationships;
- Evidence quotations;
- Uncertainty notes;
- Analysis boundaries;
- Report consistency.

Other M1 Work

Continue implementing:

- Classroom rhythm curves;
- End-of-lesson acceleration detection;
- Choral versus individual response identification;
- Generic comprehension-check detection;
- More detailed classroom structure;
- PDF export;
- Live-chat CSV import;
- Slide-page association;
- Classroom screenshots as supporting evidence.

5.2 M2: Compare Whether the Second Lesson Improved

M2 will convert teacher-confirmed suggestions into specific actions for the next lesson.

Current finding

- Teacher-confirmed issue
- Planned action for the next lesson
- Classroom behaviour to observe
- Corresponding evidence in the next recording

After the same course is recorded again, the system will compare whether:

- The teacher implemented the previous improvement;
- Wait time after questions changed;
- Students received more opportunities to respond;
- Teacher feedback changed;
- Information density or lesson structure changed;
- The change appeared in a comparable teaching context;
- The available evidence is sufficient to support an improvement claim.

M2 will compare similar teaching activities rather than only total lesson metrics.

M2 Memory

M2 will store:

- Teacher-confirmed issues;
- Accepted suggestions;
- Planned actions;
- Metrics to observe;
- Links between the two lesson rounds;
- Teacher confirmation of the second-round result.

M2 RAG

A lightweight RAG layer can support:

- Retrieval of similar teaching contexts from the previous lesson;
- Retrieval of historical evidence related to an improvement goal;
- Matching similar questions, exercises, and summaries across lessons;
- Traceable cross-lesson comparison.

5.3 M3: Multi-Course Management and Aggregated Analysis

After M1 and M2 are stable, M3 can expand to:

- Multiple course lists;
- Multiple lesson recordings for the same course;
- Cross-lesson comparison;
- Long-term teacher improvement trends;
- Course-level summary reports;
- Multi-dimensional filtering and retrieval.

The first M3 version should remain focused and should not immediately become a school-wide management platform.

M3 Memory

M3 can maintain long-term teaching-development memory, including:

- Repeated issues across lessons;
- Completed improvements;
- Long-standing unresolved issues;
- Teacher preferences for suggestion types;
- Teaching strategies used in different courses;
- Confirmed strengths;
- Reflection goals at different stages.

M3 RAG

M3 RAG can cover:

- Multiple courses;
- Multiple lesson transcripts;
- Historical evidence;
- Historical reports;
- Teacher-confirmed suggestions;
- Slides and teaching materials;
- Teaching standards and evaluation frameworks.

Retrieved results must retain source, timestamp, and course information.

5.4 Optional Extension: Mind Maps and Knowledge Graphs

After the main reflection and cross-lesson workflows are stable, the system can generate:

- Classroom knowledge mind maps;
- Knowledge-point relationship graphs;
- Teaching-activity graphs;
- Question-response graphs;
- Error-type and knowledge-point graphs.

Every node and relationship should:

- Link back to the original transcript;
- Link to the corresponding video timestamp;
- Display its source;
- Be editable by the teacher;
- Be saved only after teacher confirmation.

6. Testing Record

6.1 Current Test Coverage

Two full workflow tests have been completed:

- One recorded online lesson;
- One offline classroom video.

Both tests covered:

```
Create lesson
→ Upload video
→ Generate transcript
→ Edit transcript
→ Translate selected content
→ Calculate metrics
→ Generate teaching evidence
→ Review evidence
→ Generate report
```

Both lesson types reached the report stage.

Systematic testing has not yet been completed, including:

- Large-scale video testing;
- Multi-format compatibility testing;
- Concurrent processing;
- Long-running stability testing;
- Cross-browser testing;
- Full recovery testing;
- Offline Agent evaluation;
- Automated regression testing;
- Stress testing;
- Model-cost testing.

The current tests demonstrate that the core workflow runs, but they do not establish production-level reliability.

6.2 Recorded-Lesson Test

The recorded-lesson test focused on:

- Video upload;
- ASR transcription;
- Timestamped transcripts;
- Semantic lesson sections;
- Transcript editing;
- Translation;
- Information-density prompts;
- Recorded-lesson analysis boundaries;
- Evidence review;
- Report generation.

The test specifically checked whether the system incorrectly generated claims about student participation or wait time.

The result showed that the system mainly generated evidence related to lesson structure, language, information density, and teaching content.

6.3 Offline-Classroom Test

The offline-classroom test focused on:

- Teacher and student speech transcripts;

- Teacher-question detection;
- Wait time after questions;
- Student responses;
- Teacher feedback;
- Video timestamp navigation;
- Transcript editing;
- Candidate evidence;
- Evidence editing and confirmation;
- Report generation.

The question–wait–response–feedback chain entered downstream analysis successfully, but speaker-identification accuracy remained the main limitation.

6.4 Common Failure Case: Inaccurate Speaker Identification

Failure Behaviour

ASR speaker diarisation may:

- Classify teacher speech as student speech;
- Classify student speech as teacher speech;
- Split one student into multiple speakers;
- Merge several students into one speaker;
- Fail to determine whether a response is choral or individual.

Causes

Speaker identification is affected by:

- Microphone placement;
- Weak student voices;
- Classroom echo;
- Simultaneous speech;
- Teacher–student overlap;
- Limited ASR diarisation capability;
- Lack of additional identity data.

Impact

Speaker errors affect:

- Question-response matching;
- Student response counts;
- Teacher self-answer detection;
- Wait-time calculation;
- Feedback-target identification;
- Evidence confidence.

Current Mitigation

The system currently:

- Avoids using speaker labels as the only signal;
- Uses context, question relationships, and language behaviour;
- Adds uncertainty notes when identification is unreliable;
- Avoids strong conclusions for low-confidence segments;
- Preserves quotations and timestamps for teacher verification;
- Allows transcript correction to reduce downstream errors.

Remaining Work

Future work includes:

- Manual speaker-role editing;
- Choral versus individual response detection;
- Overlapping-speech processing;
- Speaker-confidence display;
- Automatic reanalysis after speaker correction;
- Dedicated offline test segments.

7. AI Collaboration

7.1 Intelligent Capabilities in the Product

The intelligent capabilities used during product operation include:

- Transcript Agent;
- Teaching Evidence Agent;
- External ASR;

- External large language models.

The Transcript Agent structures transcripts, while the Teaching Evidence Agent generates classroom evidence. ASR provides speech recognition, and the language model supports semantic processing, translation, and text generation.

Metrics, workflow states, and structural validation are handled by programmatic modules.

7.2 AI-Assisted Development Work

AI assisted with:

Research

- Classroom-reflection product workflows;
- Differences between video analysis and text summarisation;
- ASR, object storage, and background-task approaches;
- Agent, Workflow, RAG, and Memory use cases.

Solution Design

- M1, M2, and M3 scope planning;
- Frontend, API, Worker, data, and model-layer separation;
- Workflow design;
- Transcript display and analysis structures;
- Evidence data structures;
- Lesson-format capability boundaries.

Code Development

- TypeScript types;
- Interface and schema drafts;
- Frontend, API, and Worker code;
- Third-party service adapters;
- Repetitive refactoring work.

Debugging and Testing

- TypeScript compilation errors;
- Interface consistency;

- Environment and Secret configuration;
- Model-call errors;
- Test-case generation;
- Speaker-identification failure cases;
- Unsupported or overextended model conclusions.

7.3 Human Review and Modification

All AI-generated code, rules, and text were reviewed and modified manually.

I made the final decisions regarding:

- Actual completion status;
- M1, M2, and M3 boundaries;
- Technology choices and module boundaries;
- Transcript Agent dual projections;
- Teaching Evidence Agent Capability Matrix and output design;
- How transcript edits affect downstream analysis;
- Analysis limits for different lesson formats;
- Whether evidence matches the real video;
- Whether suggestions are appropriate for teachers;
- Whether the code compiles and runs;
- Whether cloud services are connected correctly;
- Whether test results support a completion claim.

The final product is based on manually reviewed code and real workflow tests.

8. Individual Contribution

8.1 Project Responsibility

The project was completed independently.

I was responsible for:

- Scope definition;
- User-needs analysis;
- Product-flow design;
- Frontend development;
- Backend API development;

- Worker development;
- Database design;
- Object-storage integration;
- ASR and language-model integration;
- Transcript Agent design and implementation;
- Teaching Evidence Agent design and implementation;
- Metric calculation;
- Evidence-review workflow;
- Report generation;
- Cloud deployment;
- Workflow testing;
- Technical documentation.

8.2 Main Design and Implementation Contributions

Canonical Transcript and Dual Projections

I designed one standardised transcript source with separate display and analysis projections.

This keeps teacher-visible content and downstream analysis consistent while preserving timestamps, speaker data, and classroom signals.

Semantic Lesson Sections

I transformed fragmented ASR output into semantically meaningful lesson sections with teaching-oriented titles.

This reduces the burden of sentence-by-sentence transcript review.

Teacher Revisions in Downstream Analysis

Teacher edits are used in subsequent metric and evidence analysis rather than being limited to display changes.

Capability Matrix

I designed different analysis boundaries for offline, live online, and recorded lessons.

Question–Wait–Response–Feedback Chain

I modelled classroom interaction as a connected sequence rather than relying only on question counts.

Teacher-Facing Translation

I converted internal analysis into classroom observations, teaching implications, actionable suggestions, and uncertainty notes.

Extensible Modular Architecture

The interface, API, background tasks, database access, external services, intelligent analysis, metrics, and validation are separated into modules.

This structure supports future M2 Memory, RAG, and lesson comparison, as well as M3 multi-course management.

8.3 Main Challenges

The main development challenges included:

- Large video uploads;
- State consistency between the browser, database, and object storage;
- Fragmented ASR output;
- Speaker-identification errors caused by noise and overlapping speech;
- Invalidating downstream analysis after transcript edits;
- Unsupported model conclusions;
- Different analysis boundaries for different lesson formats;
- Coordinated deployment of Web, API, and Worker services;
- Multi-cloud configuration and debugging;
- Limited development time;
- Limited real classroom data and manual labels.

8.4 Final Outcome

The project currently provides an operational classroom-reflection workflow:

Real classroom video

- Video and audio upload
- Real ASR
- Timestamped transcript
- Teacher review
- On-demand translation
- Basic metrics
- Question–wait–response–feedback analysis
- Candidate teaching evidence
- Teacher acceptance, editing, or rejection
- Confirmed classroom report

One recorded online lesson and one offline classroom lesson have each completed the full workflow.

The core workflow is operational. However, systematic testing, the AI conversation area, an independent Report Agent, Observability, Evaluation, PDF export, and several advanced classroom-analysis capabilities remain incomplete because of limited individual development time, resources, and test data.

Future work will first complete M1 reliability, reporting, and evaluation before moving to M2 cross-round improvement analysis and M3 multi-course management, Memory, RAG, and long-term teaching reflection.